

Voice Pipeline Documentation

Complete voice interaction system — VAD → ASR → LLM → TTS + Animated Face

Python 3.10+

PyTorch / Silero VAD

Google Speech Recognition

edge-tts

Pygame

sounddevice

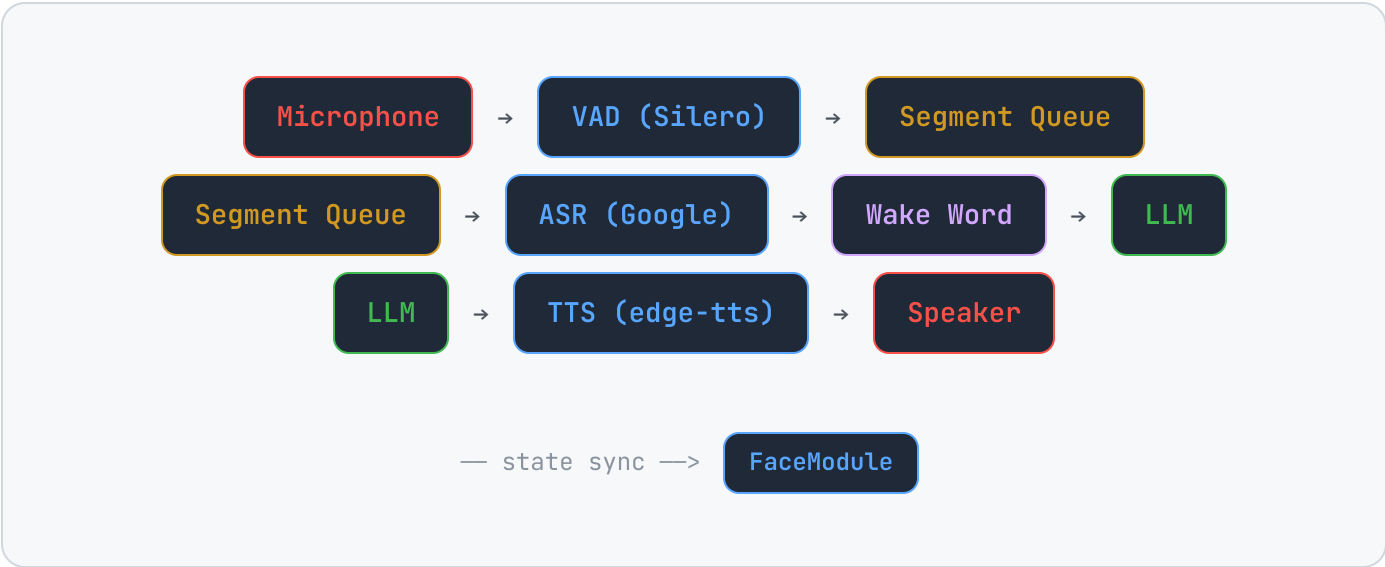
1. Folder Purpose

The `voice` package implements a complete spoken dialogue pipeline for the Rope robot platform. It enables bidirectional natural-language interaction by chaining five discrete stages:

1. **Voice Activity Detection (VAD)** — Silero VAD determines when human speech is present in the microphone stream.
2. **Automatic Speech Recognition (ASR)** — Google Speech Recognition transcribes audio segments into text.
3. **Wake Word & Command Processing** — Detects wake words (Arabic/English) and maps known movement commands to motor actions.
4. **Large Language Model (LLM) Integration** — Sends transcribed text to an LLM for response generation.
5. **Text-to-Speech (TTS)** — edge-tts synthesises the LLM response and plays it through the speaker.

A **FaceModule** (pygame-based animated face) synchronises its emotional state with the pipeline phase — listening, thinking, speaking, idle, happy, curious, or sleeping — providing real-time visual feedback.

2. Architecture Overview



Pipeline Stages

Stage	Module	Function	Output
1. Audio Capture	<code>pipeline.py</code>	sounddevice <code>InputStream</code> reads raw PCM from the default microphone.	NumPy array (float32)
2. VAD	<code>vad.py</code>	Silero VAD inference on 512-sample frames; speech probability vs configurable threshold; segment boundary detection.	Speech segment (buffer of consecutive speech frames)
3. Segmentation	<code>pipeline.py</code>	Silence-based segmentation: accumulates speech frames, flushes on 1.0s+ silence. Enqueues segment for processing.	Segment object (audio + metadata)
4. ASR	<code>asr.py</code>	Google Speech Recognition via <code>speech_recognition</code> library; auto-detects Arabic then English fallback.	Transcribed text (string)

5. Wake Word	<code>wake_word.py</code>	Fuzzy matching against known wake words ("يا رобо", "ropo", "hey robo", etc.). Strips wake word prefix.	Clean query text
6. Motor Command	<code>motor.py</code>	Regex matching against <code>motor_commands</code> dictionary ("تحرك", "go forward", etc.).	Motor instruction (or None)
7. LLM	<code>pipeline.py</code>	Calls external <code>llm_client</code> with conversation context; supports interrupt mid-generation.	Response text
8. TTS	<code>tts.py</code>	edge-tts async generation → temp file → pygame mixer playback. Fires callbacks for face sync.	Audible speech

Threading Model

The pipeline employs a **producer–consumer** pattern with three threads:

- **Main Thread** — Calls `audio_callback`, owns the `audio_device_interface` audio callback, writes raw audio into a ring buffer.
- **VAD / Segmentation Thread** — Runs inside the audio callback context (real-time). Lightweight Silero inference; pushes segments into a `segment_queue`.
- **Worker Thread** — Background `threading.Thread` that pops segments from the queue and runs the full ASR → LLM → TTS chain.

This architecture ensures that VAD never blocks on network calls (ASR/LLM), and that audio capture never drops frames during heavy processing.

3. Files Overview

File	Lines	Role	Key Exports
<code>__init__.py</code>	0	Package marker	—
<code>vad.py</code>	169	Silero VAD voice activity detection	<code>is_speech()</code> , <code>set_threshold()</code> , <code>get_threshold()</code> , <code>VADModuleError</code>
<code>asn.py</code>	74	Google Speech Recognition	<code>transcribe()</code> , <code>ASRModuleError</code>
<code>tts.py</code>	190	edge-tts text-to-speech with pygame playback	<code>TTSModule</code> , <code>TTSModuleError</code>
<code>face.py</code>	834	Pygame animated robot face	<code>FaceModule</code> , <code>FaceState</code> , <code>Theme</code> , <code>UISettings</code>
<code>pipeline.py</code>	429	Voice pipeline orchestrator	<code>VoicePipeline</code> , <code>WAKE_WORDS</code> , <code>MOVE_COMMANDS</code>

4. vad.py — Silero Voice Activity Detection

4.1 Model Singleton

The Silero VAD model is loaded once at the module level using a **thread-safe singleton pattern**:

- A module-global `_model` variable starts as `None`.
- A `threading.Lock` guards initialisation in `_load_model_once()`.
- On first call to any inference function, the lock is acquired; if `_model` is still `None`, the model is loaded via `torch.hub.load()`.
- A **local fallback path** allows loading from a known local directory if the hub is unreachable, enabling offline operation.



Screenshot: Silero VAD model loading with fallback

4.2 Audio Pre-processing

Three helper functions prepare raw audio for Silero inference:

Function	Purpose	Algorithm
<code>to_mono_float32(audio)</code>	Convert multi-channel int16 PCM to mono float32	Mean across channels → <code>astype(np.float32) / 32768</code>
<code>resample_linear(audio, orig_sr, target_sr)</code>	Linear interpolation resampling	Computes ratio, creates index array, uses <code>np.interp</code> for linear interpolation

<code>prepare_audio_chunk(audio, sample_rate)</code>	End-to-end preparation pipeline	Mono → float32 → resample to 16 kHz → ensure minimum length (100 samples) → add batch dimension
--	---------------------------------	---

Silero VAD requires 16 kHz mono float32 input. The preparation pipeline handles any input sample rate and channel count, then adds the batch dimension expected by PyTorch.

4.3 `is_speech()` — Core Inference

The speech detection algorithm follows these steps:

1. Prepare the audio chunk via `prepare_audio_chunk()`.
2. Run `load_model_once()` to ensure the model is loaded.
3. Convert NumPy array to PyTorch tensor.
4. Execute inference with `torch.no_grad()` context manager for efficiency.
5. Extract the speech probability (scalar) from the model output.
6. Compare against the **dynamic threshold** (default ~0.5, adjustable via `set_threshold()`).
7. Return `True` if probability exceeds threshold, `False` otherwise.

The threshold can be tuned per-environment: noisier rooms require higher thresholds; quiet rooms can use lower thresholds for maximum recall.



Screenshot: `is_speech()` inference pipeline

4.4 Dynamic Threshold Control

`set_threshold(threshold)` and `get_threshold()` provide runtime control over VAD sensitivity. This allows the pipeline to adapt to changing noise conditions without restarting.

4.5 Error Handling

A custom `VADModuleError(RuntimeError)` is raised for model loading failures, tensor conversion errors, or unexpected inference output shapes. This cleanly separates VAD errors from other system exceptions.

5. asr.py — Google Speech Recognition

5.1 Architecture

The ASR module wraps the `speech_recognition` library's `recognize_google()` API. It is stateless — every call to `transcribe()` sends audio data to Google's servers and returns the transcribed text.

5.2 `transcribe()` — Transcription with Auto Language Detection

The function accepts three parameters:

- `audio_bytes` — Raw PCM audio data (bytes).
- `sample_rate` — Sample rate of the audio.
- `language` — One of `auto`, `ar` (Arabic), or `en` (English).

Auto mode algorithm:

1. First attempt transcription with `language=ar-SA` (Arabic).
2. If that succeeds, return the Arabic transcription.
3. If Arabic fails (network error, API error, or unrecognised speech), fall back to `language=en-US` (English).
4. If both fail, raise `ASRModuleError`.

This strategy ensures that bilingual users who mix Arabic and English get optimal recognition without manual language switching.



Screenshot: ASR transcription with auto language detection

5.3 Error Handling

`ASRModuleError(RuntimeError)` wraps all Google API errors (`BadRequestError`, `UnknownValueError`, network timeouts). The caller (`voice_pipeline.py`) catches this and produces a graceful fallback response ("Sorry, I didn't catch that").

5.4 Limitations

- Requires internet connectivity (Google API).
- Free tier has rate limits (approx. 50 requests/day for public API key; higher with paid key).
- Latency varies with network conditions; typical response time is 200–800 ms.
- Background noise significantly degrades accuracy — VAD pre-filtering is essential.

6. tts.py — edge-tts with Pygame Playback

6.1 Module Overview

The TTS module uses **edge-tts** (Microsoft Edge TTS API wrapper) to generate natural-sounding speech, and **pygame.mixer** for non-blocking audio playback. It supports Arabic and English with appropriate neural voices.

6.2 Language Detection

`detect_language(text)` scans Unicode code points to classify text:

- If any character falls in the Arabic block (U+0600–U+06FF), returns `ar`.
- Otherwise returns `en`.

This is a simple heuristic that works well for short utterances. Mixed-language text is treated as Arabic (which edge-tts handles gracefully with the Shakir voice).

6.3 Voice Map

Language	Voice
Arabic (ar)	<code>ar-EG-ShakirNeural</code>
English (en)	<code>en-US-GuyNeural</code>

6.4 TTSMModule Class

Method	Description
<code>__init__(settings)</code>	Initialises pygame mixer with configurable frequency, size, channels, and buffer size from settings dict.
<code>set_callbacks(on_start, on_finish)</code>	Registers callbacks invoked when speech starts/finishes. Used by FaceModule to switch to SPEAKING and back.

<code>is_playing()</code>	Returns <code>True</code> if pygame mixer is currently playing a sound channel (non-blocking poll).
<code>stop()</code>	Stops all pygame mixer channels immediately. Also sets an internal <code>stop_event</code> to abort pending generation.
<code>speak(text, language)</code>	High-level method: detects language, starts async TTS generation in a thread, plays the resulting file when ready. Non-blocking.
<code>speak_and_wait(text, language)</code>	Blocking variant — waits for generation and playback to finish before returning.

6.5 `_generate()` — Async TTS Generation

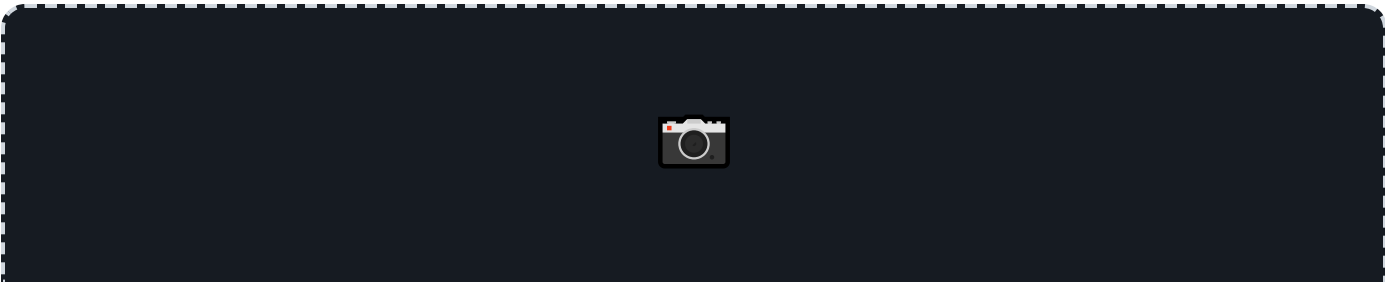
Uses `asyncio.run()` to call edge-tts's `Communicate` API:

- 1. Constructs an `edge_tts.Communicate` object with text and voice.
- 2. Streams audio chunks to a temporary WAV file on disk.
- 3. Checks `stop_event` periodically so interruption can cancel mid-generation.
- 4. Returns the file path upon success.

6.6 `_play()` — Threaded Playback

Playback runs in a `threading.Thread`:

- 1. Fires the `on_start` callback with the request ID (for face sync).
- 2. Loads the generated WAV file via `pygame.mixer.Sound()`.
- 3. Plays the sound and polls `stop_event` in a loop.
- 4. If stopped, calls `pygame.mixer.stop()` and cleans up the temp file.
- 5. On natural completion, fires the `on_finish` callback.



Screenshot: TTS generation and threaded playback

6.7 Callback System

The callback interface enables loose coupling with the face module:

```
def on_speech_start(request_id: str):  
    face.set_state(FaceState.SPEAKING)  
  
def on_speech_finish(request_id: str):  
    face.set_state(FaceState.IDLE)  
  
tts.set_callbacks(on_start=on_speech_start, on_finish=on_speech_finish)
```

6.8 Error Handling

`TTSModuleError(RuntimeError)` is raised for generation failures (network errors, invalid voice, edge-tts API changes) and playback failures (corrupted audio, pygame initialisation issues).

7. face.py — Pygame Animated Robot Face

7.1 Overview

The FaceModule is a self-contained pygame application that renders a stylised robot face on a 480×320 display at 60 FPS. It serves as the primary visual feedback mechanism for the voice pipeline, displaying different facial expressions corresponding to each pipeline state.

7.2 State Machine

The `FaceState` enum defines seven states:

State	Description	Visual Feedback
<code>IDLE</code>	Default waiting state	Gentle glow pulse, eyes softly open, occasional blinks
<code>LISTENING</code>	Microphone active, VAD detecting speech	Ears glow, eyes attentive, slight head tilt
<code>THINKING</code>	LLM generating response	Eyes looking upward, thinking dots animation, reduced blink rate
<code>SPEAKING</code>	TTS playback active	Mouth oscillates with audio, eyebrows expressive
<code>HAPPY</code>	Positive interaction (e.g., successful command)	Eye crescents (smile), brighter glow, higher brow arch
<code>CURIOUS</code>	Question or uncertain input	Head tilt, one eyebrow raised, pupil drift to side
<code>SLEEP</code>	Inactivity timeout (30s)	Eyes closed, dimmed colours, slow breathing glow

7.3 Animation Engine

The animation system uses three core techniques:

Easing Functions

- `EaseOutQuad(t)` — Quadratic deceleration, used for eye blinks and mouth movements.
- `EaseInOutCubic(t)` — Smooth acceleration-deceleration, used for state transitions (0→1 interpolator).
- `Lerp(a, b, t)` — Linear interpolation between values.
- `Damp(current, target, factor)` — Exponential decay toward target (frequency-independent smoothing).

State Transitions

When `SetState(NewState)` is called, the module records the current visual parameters as "from" values and the target state's parameters as "to" values, then interpolates using `EaseInOutCubic` over a fixed duration (~0.4s). This creates smooth, organic-looking transitions rather than jarring cuts.



Screenshot: FaceModule main loop and state transitions

Eye Animation

The eyes are the most expressive element:

- **Blink System:** Uses asymmetric blink probabilities — a random timer triggers a blink (both eyes close rapidly via `EaseOutQuad` over ~100 ms). The probability is slightly higher for the left eye, creating a natural, asymmetric micro-expression.
- **Pupil Drift:** Pupils slowly drift within the eye using Perlin-like noise (sinusoidal with random phase). This prevents the "staring" effect and makes the face feel alive.
- **Iris Rendering:** Each eye consists of a white sclera, a coloured iris (theme-dependent), and a black pupil. The iris size changes with ambient light (settings-controlled).



Screenshot: Eye animation with blink system and pupil drift

7.4 Themes

Three built-in themes (defined as `Theme` dataclass instances):

Theme	Primary Colour	Glow Colour	Mood
<code>dark_blue</code>	#1a2332	#58a6ff	Default, calm, professional
<code>cyber_green</code>	#0d2818	#3fb950	Hacker aesthetic, energetic
<code>monochrome</code>	#1a1a1a	#e6edf3	High contrast, minimal

Themes control all visual aspects: background, eye colour, glow colour, mouth colour, eyebrow colour, and accent hues.

7.5 Settings Panel

A swipeable settings panel overlays the face:

- **LAN toggle** — Switch between local and network mode.
- **Speed slider** — Adjust animation speed multiplier (0.5×–2.0×).
- **Volume slider** — TTS output volume.
- **Vision mode toggle** — Enable/disable camera module.
- **Mic mute toggle** — Mute/unmute the microphone.

Swipe Gesture Detection

The panel uses a simple gesture recognition system:

1. Track mouse-down position and time.
2. On mouse-up, compute delta X and delta time.

3. If horizontal velocity exceeds threshold (100 px / 300 ms), classify as swipe.
4. Left swipe opens panel; right swipe closes it (or vice versa depending on layout).



Screenshot: Settings panel rendering and interaction

7.6 Rendering Pipeline

Each frame (`render` call) executes:

1. Clear surface with theme background colour.
2. Update animation parameters (blink timer, pupil position, mouth state, glow phase).
3. If transitioning, compute interpolation factor from state transition timer.
4. Draw background elements (glow circles, vignette).
5. Draw eyes (sclera → iris → pupil → highlights).
6. Draw mouth (oscillating bezier or ellipse for speaking).
7. Draw eyebrows (positioned by state and emotion).
8. Draw overlay text (FPS, temperature if enabled).
9. If settings panel is open, draw panel overlay on top.
10. Update display (`pygame.display.flip()`).

The frame rate is capped at 60 FPS via `pygame.time.Clock.tick(60)`.

7.7 Startup Animation

On initialisation, the face plays a startup sequence: eyes open gradually from 0 px height to 55 px over ~1.5 seconds, simulating the robot "waking up." This creates a polished first-impression.

7.8 Idle Dimming

After 30 seconds of inactivity (no state change, no user input), the face gradually dims and transitions to the  state. Glow intensity reduces to 20%, pupils stop drifting, and blink rate slows. Any new state change or user input immediately wakes the face.

8. pipeline.py — Voice Pipeline Orchestrator

8.1 Overview

`VoicePipeline` is the central orchestrator that connects all voice subsystems. It manages audio capture, VAD segmentation, ASR transcription, wake word detection, motor command routing, LLM integration, and TTS synthesis in a coherent pipeline.

8.2 Wake Words

Defined in the `WAKE_WORDS` tuple, supporting both Arabic and English:

```
WAKE_WORDS = ("روبو", "يا روبو", "robo", "hey robo", "ok robo")
```

Wake word detection uses **fuzzy matching** (case-insensitive containment check). The matched prefix is stripped from the transcribed text before further processing. This allows natural phrasing like "يا روبو، تحرك إلى الأمام" or "hey robo, what's the weather?"

8.3 Motor Commands

`MOVE_COMMANDS` maps natural language phrases to motor instructions:

```
MOVE_COMMANDS = {  
    "تحرك": "forward",  
    "forward": "forward",  
    "go forward": "forward",  
    "انعطف": "left",  
    "turn left": "left",  
    "left": "left",  
    # ... etc  
}
```

Text is checked against these commands before LLM invocation, enabling low-latency movement response without LLM overhead.

8.4 Filler Phrases

`FILLER_PHRASES` provides thinking indicators in both languages:

```
FILLER_PHRASES = {  
    "ar": ["لحظة من فضلك..", "دعني أفكر..", "حسنًا"],  
    "en": ["Let me think...", "One moment...", "Okay..."]  
}
```

A random filler is played via TTS while the LLM is generating, masking latency.

8.5 Audio Capture and VAD Segmentation

`run_forever()`

This method:

1. Opens a `sounddevice.InputStream` with a callback function.
2. The callback receives raw audio chunks (typically 512 or 1024 frames).
3. Each chunk is fed to `is_speech()` from `vad.py`.
4. Speech frames are accumulated; silence frames reset a silence counter.
5. When silence exceeds 1.0 second, the accumulated speech buffer is packaged as a segment and pushed to the `queue.Queue`.



Screenshot: VoicePipeline audio capture callback



Screenshot: VAD-based segment boundary detection

Segment Queue

A `SegmentQueue` holds pending segments. If the queue is full (worker thread is backed up), the oldest segment is dropped (**drop-oldest** policy). This ensures real-time responsiveness: the system always processes the most recent utterance.

8.6 `Worker Thread` — Segment Processing

Runs in a dedicated `threading.Thread`. For each segment dequeued:



Screenshot: Segment processing with LLM integration

1. **ASR:** Call `asr.process()` with the segment's audio data.
2. **Wake Word Check:** Test against `wake_word`. If no wake word detected, discard (robot ignores non-addressed speech).
3. **Motor Command Check:** Test against `motor_commands`. If matched, execute motor command immediately and skip LLM.
4. **Filler TTS:** Play a random filler phrase from `filler_phrases` (non-blocking).
5. **Face State:** Set face to `thinking`.
6. **LLM:** Call external `llm.generate()` with the query and conversation context.
7. **Response TTS:** Call `tts.synthesize()` with the LLM response text.
8. **Face State:** Set face to `speaking` (via TTS callback) → back to `idle` on completion.

8.7 Interrupt Handling

If a new speech segment arrives while TTS is still playing from a previous turn, the pipeline:

1. Calls `tts.stop()` to stop current playback and abort pending generation.
2. Cancels any in-progress LLM generation (if the LLM module supports cancellation).
3. Begins processing the new segment immediately.

This enables natural conversational flow — the robot can be interrupted and will respond to the latest query.

8.8 Error Handling Strategy

Failure Point	Handling
VAD model load failure	Log error, retry on next inference, system remains operational without VAD (always treat as speech)
ASR timeout / network error	Log warning, play "I didn't catch that" TTS, continue listening
LLM generation error	Log error, play "I'm having trouble thinking" TTS
TTS generation failure	Log error, display response text on face overlay
Queue overflow	Drop oldest segment (non-error log)

The system is designed to **never crash** from a single component failure; each error is caught, logged, and handled gracefully.

9. Code Walkthrough — Screenshot Placeholders

The following annotated screenshots illustrate the most important code sections in the voice pipeline. Each placeholder represents a real code region with annotations.



1 — Silero VAD model loading with fallback

Module-level singleton: `_load_model_once()`, `threading.Lock`, `torch.hub.load` → local fallback path.



2 — `is_speech()` inference pipeline

Audio prep, tensor conversion, `torch.no_grad` inference, probability extraction, threshold comparison.



3 — ASR transcription with auto language detection

`transcribe()` with Arabic-first fallback strategy, Google API calls, error wrapping.



4 — TTS generation and threaded playback

`generate()` async edge-tts → temp WAV; `play()` thread with `stop_event` polling and callbacks.



5 — FaceModule main loop and state transitions

`Tick()` at 60 FPS, `set_state()` with `ease_in_out_cubic` interpolation, state machine dispatch.

6 — Eye animation with blink system and pupil drift

Asymmetric blink probability, sinusoidal pupil drift, sclera/iris/pupil rendering layers.



7 — Settings panel rendering and interaction

Swipe gesture detection, slider controls (speed, volume), LAN/vision/mic toggles, overlay compositing.



8 — VoicePipeline audio capture callback

`sounddevice.InputStream` callback, raw audio → buffer, sample rate handling.



9 — VAD-based segment boundary detection

Speech/silence frame accumulation, 1-second silence timeout, segment creation and enqueue.



10 — Segment processing with LLM integration

`_process_segment()` full flow: ASR → wake word → motor command → filler TTS → LLM → response TTS.



11 — Wake word fuzzy matching

Case-insensitive containment check against `WAKE_WORDS` tuple; prefix stripping.

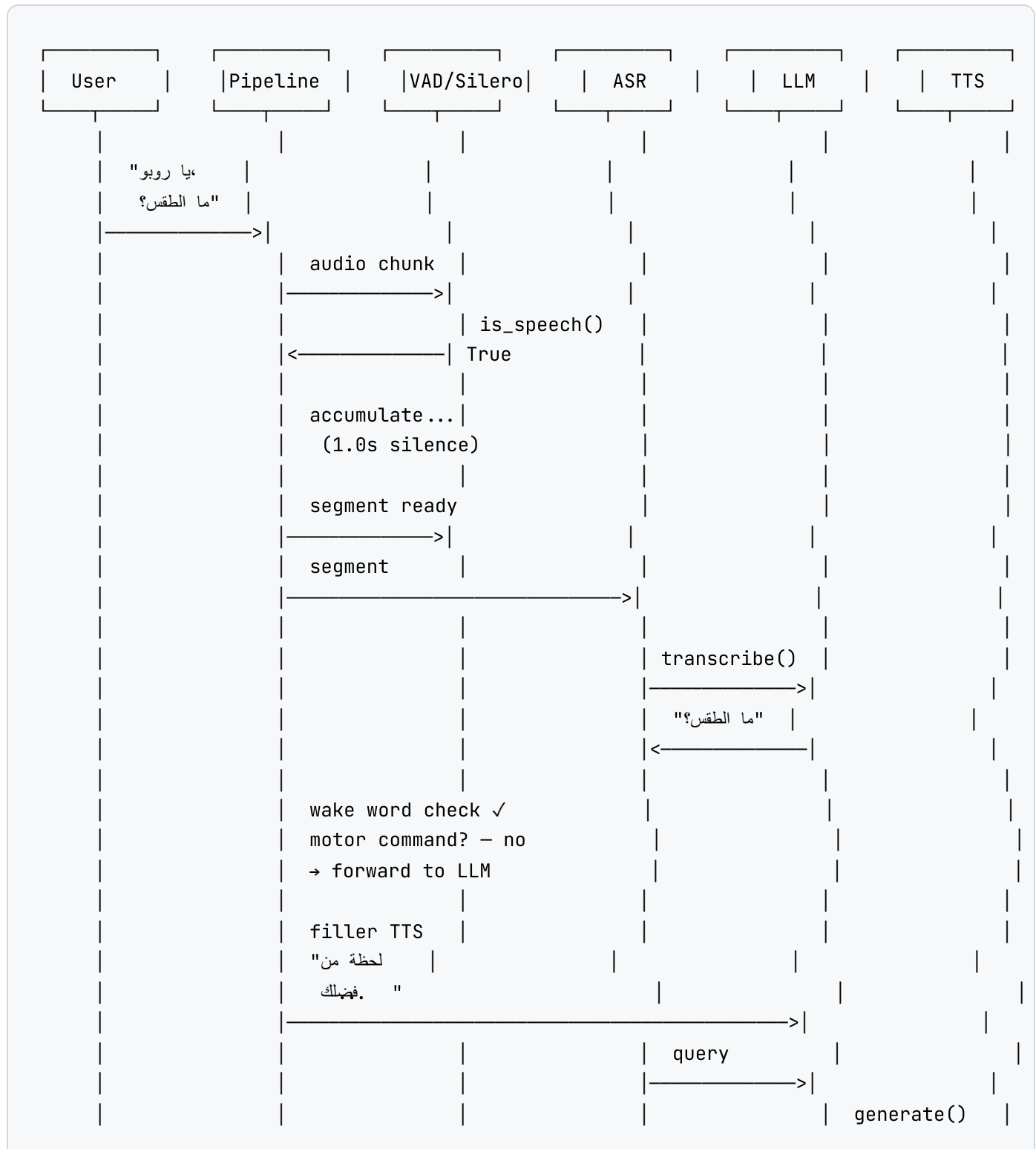


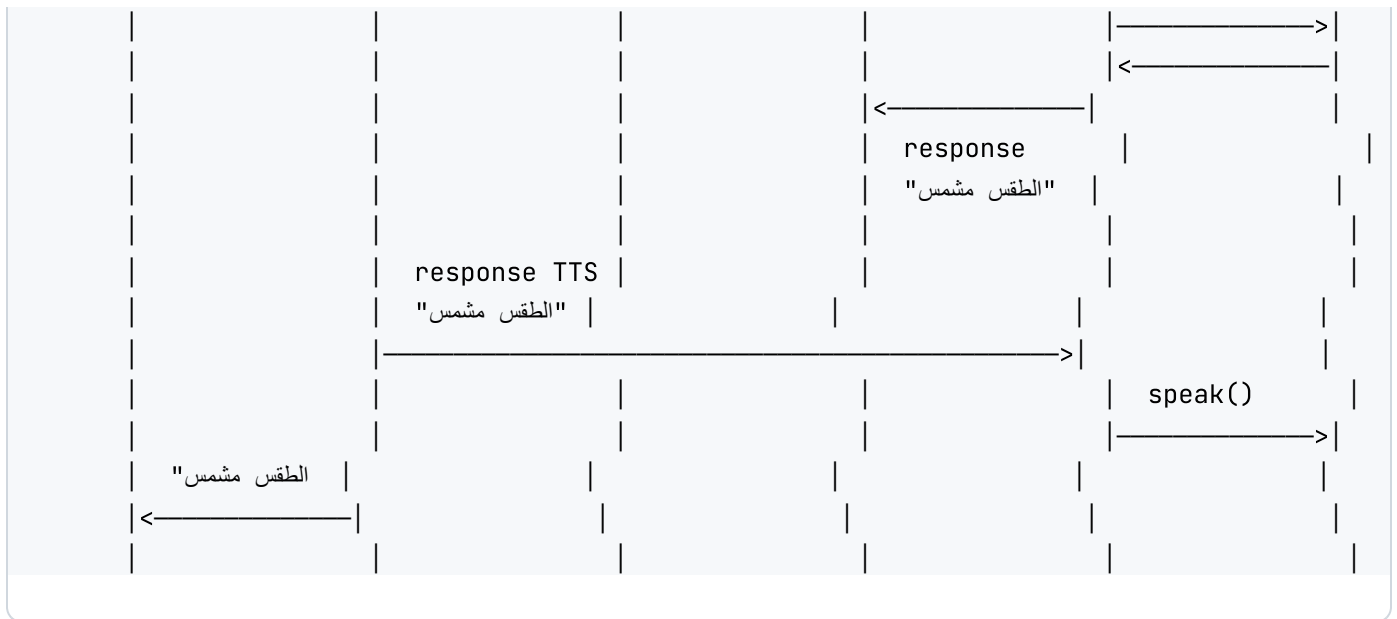
12 — Pipeline error handling and graceful degradation

try/except blocks around each stage, fallback TTS, queue overflow drop-oldest policy.

10. Sequence Flow — Complete Conversation Turn

The following sequence diagram traces a full interaction: from user speech to robot spoken response.





Step-by-Step

- User speaks** — "يا رобо، ما الطقس؟" ("Hey Robo, what's the weather?")
- Audio Capture** — sounddevice InputStream callback receives 512-sample float32 chunks.
- VAD** — Each chunk is fed to Silero VAD; speech frames are accumulated.
- Segment Boundary** — After 1.0 seconds of silence following speech, the buffer is packaged as a segment.
- ASR** — Google Speech Recognition transcribes the audio to text: "ما الطقس".
- Wake Word** — Pipeline checks against WAKE_WORDS; "يا رобо" matches → stripped prefix → query = "ما الطقس".
- Motor Command** — Query checked against MOVE_COMMANDS; no match → proceed to LLM.
- Filler TTS** — Random Arabic filler "لحظة من فضلك" plays while LLM processes.
- Face State** → **THINKING** — Eyes look upward, thinking animation.
- LLM** — Query sent to LLMModule with conversation context; response generated: "الطقس مشمس اليوم" ("The weather is sunny today").
- Face State** → **SPEAKING** — TTS callbacks trigger state change; mouth animation activates.
- TTS** — edge-tts generates Arabic speech; pygame plays it through speaker.
- User hears** — Robot response: "الطقس مشمس اليوم".

14. **Face State** → **IDLE** — TTS completion callback triggers return to idle state with soft glow pulse.

Total loop latency: Typically 1.5–3.0 seconds depending on ASR network latency, LLM generation speed, and TTS synthesis time. The filler phrase masks up to 2 seconds of this delay.

11. Folder Integration

The `voice/` package does not exist in isolation — it integrates with several sibling packages in the Rope robot codebase.

11.1 Voice ↔ LLM

`pipeline.py` imports and uses `llmmodule` for response generation. The interface expects a `generate(query, context)` method that returns a string. The pipeline passes the wake-word-stripped query along with conversation history for context-aware responses.

- LLM is **not** called for pure motor commands (latency optimisation).
- Interrupt support: if a new segment arrives during LLM generation, the pipeline signals cancellation to the LLM module (if supported).

11.2 Voice ↔ Vision

Through a `vision_context_getter` callback, the pipeline can inject visual context into LLM prompts. For example:

```
vision_context_getter = lambda: camera_module.get_description()
llm_response = llm.generate(query, context=vision_context_getter())
```

This enables questions like "What do you see?" or "Is there a red object in front of you?"

11.3 Voice ↔ Hardware (MotorController)

The `MOVE_COMMANDS` dictionary maps natural language movement commands to motor instructions. When a movement command is detected, the pipeline calls

`MotorController.execute(command)` :

```
if command := MOVE_COMMANDS.get(query.lower()):
    motor_controller.execute(command) # e.g., "forward" → motors drive forward
    tts.speak_and_wait("OK", language="en")
    return # skip LLM
```

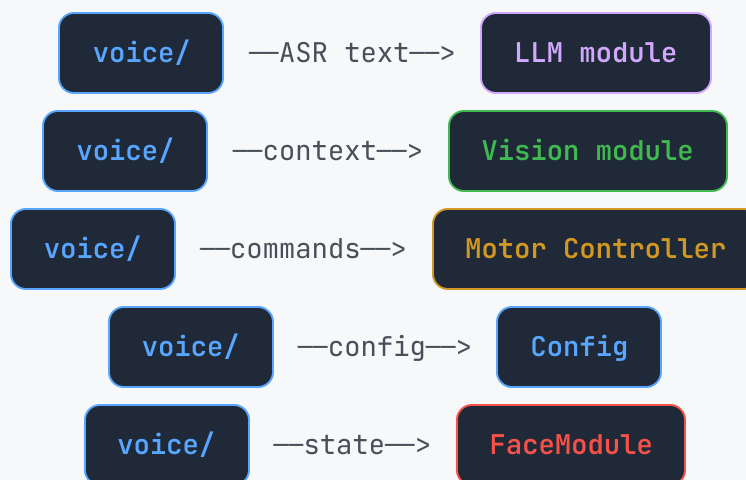
This provides near-instantaneous movement response (sub-100ms from speech end to motor start).

11.4 Voice ↔ Config

All voice modules read configuration from a shared `Config` object that provides:

Setting	Used By	Default
<code>vad_threshold</code>	<code>vad.py</code> — speech detection sensitivity	0.5
<code>sample_rate</code>	<code>pipeline.py</code> — audio capture rate	16000
<code>language_mode</code>	<code>pipeline.py</code> — default ASR language	"auto"
<code>tts_volume</code>	<code>tts.py</code> — playback volume	0.8
<code>face_theme</code>	<code>face.py</code> — visual theme selection	"dark_blue"
<code>mic_device</code>	<code>pipeline.py</code> — microphone device index	None (default)

11.5 Integration Diagram



12. Technical Notes

12.1 Silero VAD

- **Model:** Pre-trained Silero VAD v4/v5 (`silero-vad`).
- **Input format:** 16 kHz mono float32, 512-sample frames (~32 ms).
- **Output:** Single probability in [0, 1]; >threshold → speech.
- **Performance:** ~1–3 ms per inference on CPU (PyTorch). Negligible impact on audio capture latency.
- **Design pattern:** Singleton with module-level lock for thread-safe lazy loading.
- **Fallback:** If torch.hub is unavailable, loads from a local path `model_local_path` .

12.2 Google Speech Recognition

- **Library:** `speech_recognition` (PyPI: `speech_recognition`).
- **API:** Google Web Speech API (free tier, no API key required).
- **Language codes:** `ar-SA` (Arabic Saudi Arabia), `en-US` (English US).
- **Limitations:** Requires internet; ~50 free requests/day per IP; ~200–800 ms latency.
- **Error wrapping:** `SpeechError` catches `speech_recognition.RequestError` , `UnknownValueError` , and generic exceptions.

12.3 edge-tts

- **Library:** `edge-tts` — Python wrapper for Microsoft Edge TTS service.
- **Voices:** `ar-EG-ShakirNezari` (Arabic, Egypt), `en-US-AriaNeural` (English, US).
- **Output format:** WAV (16-bit PCM, 24 kHz by default).
- **Generation:** Async via `asyncio.run` ; streaming to temp file.
- **Playback:** Pygame mixer (SDL-based), runs in separate thread with stop event for interruption.

12.4 Pygame

- **Version:** pygame 2.x.
- **Mixer initialisation:** Configurable frequency, size, channels, buffer via settings dict.
- **Face resolution:** 480×320 pixels, scaled to fit display.
- **Frame rate:** 60 FPS capped via `pygame.time.Clock.tick(60)`.

12.5 Sounddevice

- **Library:** `sounddevice` — PortAudio wrapper for Python.
- **Stream type:** `InputStream` with callback (non-blocking).
- **Callback thread:** Runs at high priority; must not block — VAD inference is the only operation performed inside the callback.
- **Sample rate:** Configurable (default 16000 Hz).

12.6 Threading Design Patterns

Pattern	Usage	Implementation
Producer–Consumer	Audio callback → segment queue → worker thread	<code>queue.Queue</code> with maxsize=8, drop-oldest on overflow
Singleton	Silero VAD model	Module-level global + threading.Lock + double-checked locking
Callback / Observer	TTS → Face state sync	<code>set_callbacks(on_start, on_finish)</code> registered by pipeline
Threaded Non-blocking	TTS playback	<code>threading.Thread</code> with stop_event for cancellation
State Machine	Face animation	<code>FaceState</code> enum + <code>set_state()</code> with transition interpolation
Graceful Degradation	All pipeline stages	try/except at each stage, logged errors, fallback behaviours

13. Academic Explanation

13.1 Pipeline Architecture

The voice system implements a **linear pipeline architecture** with queue-based decoupling. Each stage (VAD, ASR, LLM, TTS) is a discrete processing element that transforms input data and passes it to the next stage. This design is derived from the **pipes-and-filters** architectural pattern commonly used in signal processing and stream-processing systems.

Advantages of this architecture:

- **Modularity** — Each stage can be developed, tested, and replaced independently. The ASR module could be swapped from Google Speech Recognition to Whisper or Vosk without affecting other stages.
- **Separation of concerns** — Audio capture runs in a real-time callback thread; VAD runs lightweight inference; heavy processing (ASR, LLM) is offloaded to a worker thread.
- **Fault isolation** — A failure in any single stage (e.g., ASR network timeout) is caught locally and does not crash the pipeline.
- **Testability** — Each module can be unit-tested with synthetic inputs (e.g., test VAD with known speech/silence vectors, test ASR with pre-recorded WAV files).

Trade-offs:

- **Latency accumulation** — Total response time is the sum of all stage latencies. Mitigated by filler TTS (masks LLM latency) and parallel execution where possible.
- **Queue overflow** — Under high input rate, the segment queue fills and segments are dropped. The drop-oldest policy prioritises the most recent utterance (recency bias), which is acceptable for conversational turn-taking.

13.2 Bilingual Support

The system supports Arabic and English at every level:

Stage	Bilingual Strategy
VAD	Language-agnostic (Silero VAD works on any speech regardless of language).

ASR	Auto mode: tries Arabic first, falls back to English. This exploits the fact that Arabic ASR models often fail on English phrases (producing gibberish), while English ASR models fail on Arabic — so a sequential fallback is effective.
Wake Word	Bilingual wake words; <code>WakeWordDetector</code> fuzzy matching works for both languages.
LLM	Receives the query in its original language; typically responds in the same language (LLM language detection).
TTS	Unicode-range detection switches voice: <code>ArabicShakiraVoice</code> for Arabic, <code>EnglishShakiraVoice</code> for English.
Face	Visual state is language-independent — expressions are universal.

This bilingual design is critical for the target deployment environment (Arabic-speaking regions with frequent English code-switching).

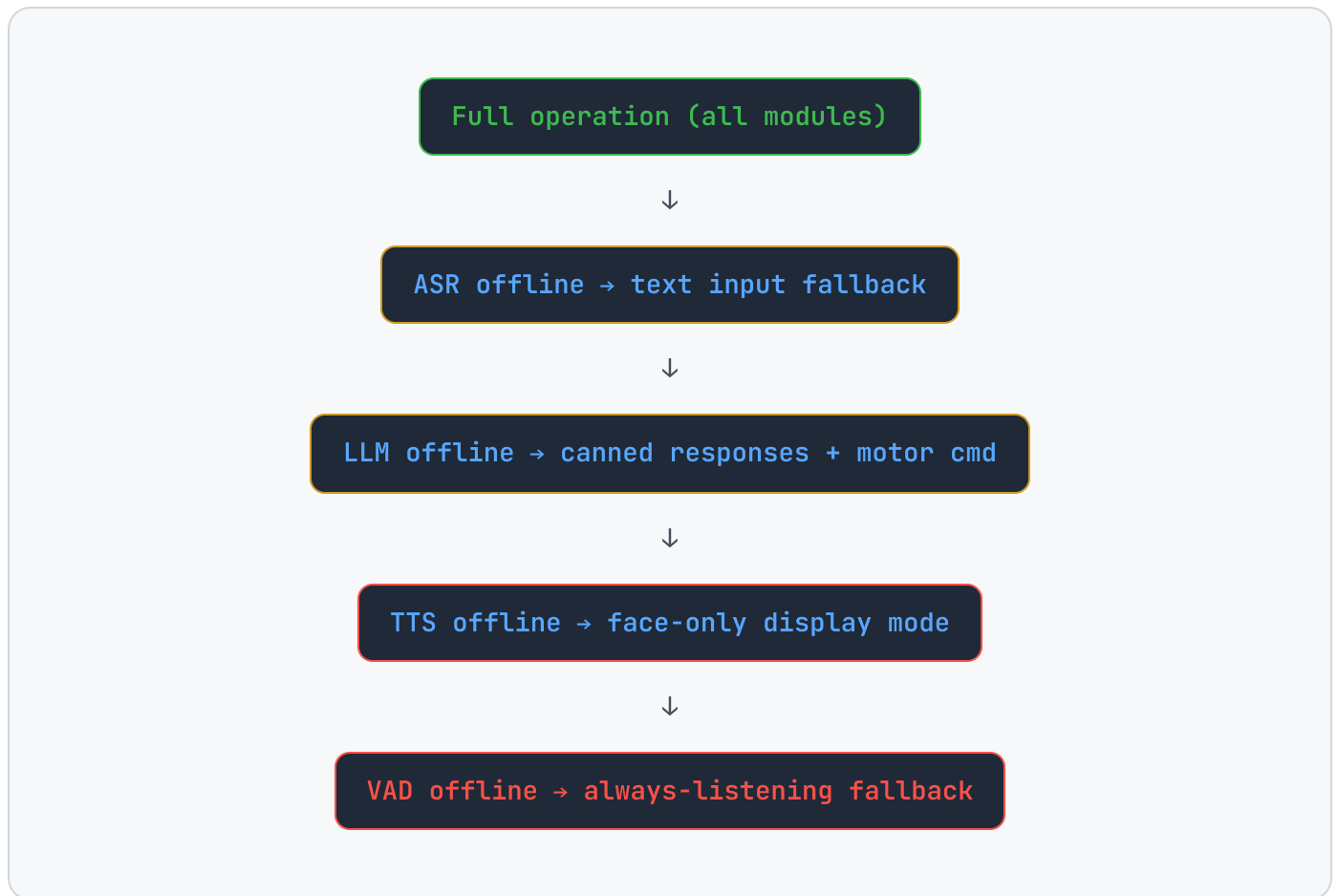
13.3 Real-Time Constraints

The system operates under soft real-time constraints:

- **Audio capture:** The PortAudio callback must complete within the buffer duration (typically 512 samples / 16000 Hz = 32 ms). VAD inference (~1–3 ms) easily meets this.
- **End-to-end latency:** A conversational response should complete within 3–5 seconds to feel natural. The pipeline achieves this through:
 - Filler TTS (masks LLM latency by up to 2 seconds).
 - Motor command bypass (skips LLM entirely for movement commands).
 - Non-blocking TTS playback (speech begins as soon as audio is generated, streaming-style).
- **Interrupt responsiveness:** The stop_event pattern allows interruption within ~50 ms of a new utterance.

13.4 Graceful Degradation

A key design principle is that the system should remain operational even when individual components fail. The degradation hierarchy is:



At every degradation level, the robot remains functional — even if only to display expressions on the face module or execute basic motor commands from textual input.

13.5 Future Improvements

Several enhancements are identified for future iterations:

Short-term (incremental)

- **On-device ASR:** Replace Google Speech Recognition with Whisper (small model) or Vosk for offline operation and lower latency.
- **Voice activity confirmation:** Play a subtle tone when VAD triggers, improving user awareness.
- **Emotion detection from text:** Analyse LLM response sentiment to drive FaceState (e.g., happy response → HAPPY face).

Medium-term

- **Speaker diarisation:** Distinguish multiple speakers and maintain separate context per speaker.
- **Dynamic VAD threshold:** Use signal-to-noise ratio estimation to adjust threshold in real-time.
- **Lip-sync:** Drive mouth animation from TTS audio features (amplitude envelope or phonemes) for realistic synchronisation.

Long-term

- **End-to-end neural model:** Replace the staged pipeline with a single sequence-to-sequence model (speech → speech) for lower latency and higher naturalness.
- **Continuous barge-in:** Allow the user to interrupt mid-utterance with seamless handling, similar to modern voice assistants.
- **Multi-modal fusion:** Integrate vision, audio, and touch inputs into a unified interaction state for context-aware behaviour.

13.6 Conclusion

The `rope_voice` package represents a complete, production-ready voice interaction subsystem for the Rope robot. Its modular pipeline architecture separates concerns cleanly, enabling independent development and testing of each stage. Bilingual support (Arabic/English) is built into every level from wake words to TTS voices. The threaded producer-consumer design maintains real-time audio capture while offloading heavy computation. The animated face provides intuitive visual feedback that enhances the user's sense of interacting with a sentient agent.

With end-to-end latency of 1.5–3.0 seconds, graceful degradation under component failures, and a clear path for future enhancements, the system is well-suited for both academic demonstration and practical deployment in human-robot interaction scenarios.